

Curso: Estructuras de Datos

Periodo: I semestre 2026

Valor: 67 pts

Profesor (a): Raquel Mora Rojas

Nombre del Estudiante: _____ Nota: _____

Carné: _____

Instrucciones Generales:

- El examen debe realizarse de forma individual. Cualquier sospecha de fraude se asignará nota mínima en el curso.
- El tiempo para realizar la prueba es de 90 minutos a partir de la indicación del profesor (a).
- Responda lo que se le solicita de forma clara y ordenada, limite las respuestas al contenido visto en clase y material de referencia.
- Utilice lapicero azul o negro, en caso de utilizar lápiz o corrector pierde el derecho a reclamo.
- El uso de cualquier dispositivo electrónico (celulares, smartwatch, tablets, entre otros) queda prohibido durante la prueba.

I Parte – Selección Múltiple

Valor: 17 pts

Marque todas las opciones correctas por pregunta. Algunas preguntas tienen más de una respuesta correcta, lea cuidadosamente.

1. ¿Cuál es la diferencia principal entre un arreglo y una lista en términos de tamaño y flexibilidad de almacenamiento?
 - a) Los arreglos tienen un tamaño fijo y la memoria se asigna de forma estática, mientras que las listas pueden cambiar de tamaño dinámicamente y utilizan asignación de memoria dinámica.
 - b) Los arreglos son más flexibles en términos de manipulación de elementos, mientras que las listas son más eficientes en el acceso aleatorio.
 - c) Los arreglos pueden contener tipos de datos diferentes, mientras que las listas están limitadas a un solo tipo de datos.
 - d) Los arreglos permiten la inserción y eliminación en $O(1)$ en cualquier posición, mientras que las listas solo lo permiten al inicio.
2. ¿Por qué las matrices multidimensionales se almacenan en la memoria de forma continua en C++?
 - a) Para permitir un acceso eficiente a los elementos mediante cálculos de índices.
 - b) Para reducir la fragmentación de la memoria.

- c) Para facilitar la implementación de operaciones de matriz como la multiplicación.
- d) Porque el estándar de C++ no permite el uso de punteros no contiguos para matrices de tamaño fijo.

3. ¿Cuál es la principal diferencia entre una lista simplemente enlazada y una lista doblemente enlazada?

- a) Una lista doblemente enlazada permite el recorrido bidireccional, mientras que una lista simplemente enlazada solo permite recorrido en un sentido.
- b) Una lista simplemente enlazada utiliza menos memoria total que una lista doblemente enlazada, ya que los nodos son de menor tamaño.
- c) En una lista doblemente enlazada, cada nodo almacena un puntero al nodo anterior y al siguiente, mientras que en una lista simple cada nodo solo tiene un puntero al siguiente.
- d) Una lista doblemente enlazada siempre ordena sus elementos automáticamente al momento de la inserción.

4. ¿Cuál de las siguientes afirmaciones describe mejor un Tipo Abstracto de Datos (TAD)?

- a) Un modelo matemático que define la estructura de datos y las operaciones que se pueden realizar sobre ella, sin especificar cómo se implementan.
- b) Una clase de C++ que encapsula datos privados y expone métodos públicos para manipularlos.
- c) Un conjunto de datos sin procesar almacenado en la memoria de la computadora.
- d) Una implementación específica de una estructura de datos optimizada para un lenguaje de programación determinado.

5. ¿Cuáles son ventajas reales de una lista enlazada circular frente a una lista enlazada simple?

- a) Facilita la implementación de operaciones cíclicas como la rotación de elementos o la gestión de turnos entre procesos.
- b) Ocupa menos espacio en memoria que una lista enlazada simple con la misma cantidad de elementos.
- c) Garantiza que el tiempo de búsqueda de cualquier elemento sea $O(1)$ gracias a su estructura circular.
- d) Es útil en algoritmos que requieren procesar elementos de forma continua y repetitiva sin necesidad de reiniciar el recorrido manualmente.

6. ¿De qué formas contribuyen los arreglos a mejorar la eficiencia y legibilidad del código?

- a) Almacenan datos de manera ordenada y permiten acceder a ellos directamente mediante índices.

- b) Reducen la complejidad del código al agrupar datos relacionados bajo una misma variable.
- c) Eliminan la necesidad de gestión explícita de memoria al crecer automáticamente cuando se añaden elementos.
- d) Habilitan el uso de algoritmos de búsqueda y ordenamiento optimizados que requieren acceso aleatorio.

7. ¿Qué aspecto de la administración de memoria es crucial para evitar fugas de memoria en C++?

- a) Asignar la mayor cantidad de memoria posible al inicio del programa para reducir las solicitudes posteriores al sistema operativo.
- b) Liberar explícitamente la memoria asignada dinámicamente con `new` cuando ya no se necesita, utilizando `delete`.
- c) Utilizar exclusivamente memoria estática en lugar de memoria dinámica para todas las estructuras de datos.
- d) Reutilizar bloques de memoria previamente liberados antes de solicitar nuevas asignaciones al sistema operativo.

8. ¿Cuáles son desventajas reales del uso de arreglos frente a listas en términos de inserción y eliminación?

- a) Los arreglos siempre ocupan más espacio en memoria que las listas para el mismo número de elementos.
- b) La inserción o eliminación en una posición intermedia de un arreglo requiere desplazar los elementos adyacentes, lo que implica $O(n)$ en el peor caso.
- c) Los arreglos son más difíciles de declarar e inicializar que las listas enlazadas en C++.
- d) Al eliminar un elemento de un arreglo estático, el espacio liberado no puede ser reutilizado por otros datos sin reorganizar la estructura manualmente.

9. ¿Qué estrategias permiten recorrer una lista enlazada circular sin caer en un bucle infinito?

- a) Guardar un puntero al nodo inicial antes de comenzar el recorrido y detener la iteración cuando el nodo actual vuelve a ser ese mismo nodo.
- b) Verificar si el puntero siguiente del nodo actual es `NULL` antes de avanzar al siguiente elemento.
- c) Utilizar un contador que limite la cantidad máxima de elementos que pueden agregarse a la lista.
- d) Mantener un puntero adicional que apunte al último nodo de la lista y usarlo como condición de parada del recorrido.

10. ¿Cuáles de las siguientes afirmaciones sobre algoritmos de búsqueda son correctas?

- a) La búsqueda binaria requiere que la estructura de datos esté ordenada previamente y tiene una complejidad temporal de $O(\log n)$.
- b) La búsqueda secuencial puede aplicarse tanto a listas ordenadas como desordenadas, con una complejidad temporal de $O(n)$ en el peor caso.
- c) La búsqueda secuencial tiene una complejidad temporal de $O(\log n)$, lo que la hace más eficiente que la búsqueda binaria en listas pequeñas.
- d) La búsqueda binaria divide el espacio de búsqueda a la mitad en cada iteración, lo que la hace significativamente más eficiente que la búsqueda secuencial en listas grandes y ordenadas.

II Parte – Respuesta Corta

Valor: 15 pts

Lea detalladamente la pregunta y sea claro y conciso con su respuesta.

1. Explique en sus palabras qué es la abstracción y dé un ejemplo aplicado a estructuras de datos. 3 pts

2. Describa cómo funciona el algoritmo de búsqueda binaria y en qué tipo de estructuras de datos se puede aplicar de manera efectiva. 3 pts

3. ¿Qué desafíos adicionales podrían surgir al aplicar algoritmos de ordenamiento como el Quicksort o el merge sort a una lista enlazada en comparación con un arreglo estático? 3 pts

4. Enumere las características principales de una pila y una cola. Explique una ventaja de cada una de estas estructuras sobre la otra en términos de su uso práctico. 3 pts

5. Proporcione un ejemplo concreto de cómo una pila podría utilizarse en un programa de procesamiento de texto para deshacer y rehacer operaciones. 3 pts

III Parte – Desarrollo

Valor: 35 pts

Solucione lo que se le solicita en forma clara y ordenada.

Ejercicio 1

15 pts

El siguiente código intenta multiplicar dos matrices cuadradas NxN almacenando el resultado en C. Sin embargo, al comparar los valores obtenidos con los esperados, el resultado nunca es correcto. Además, al ejecutarlo se presentan fallos esporádicos. Examine el código:

```
1. #define N 3
2. void multiplicar(int A[][N], int B[][N], int C[][N]) {
3.     for (int i = 0; i < N; i++)
4.         for (int j = 0; j < N; j++)
5.             C[i][j] = 0;
6.     for (int i = 0; i < N; i++)
7.         for (int j = 0; j < N; j++)
8.             for (int k = 0; k < N; k++)
9.                 C[i][j] = A[i][k] * B[k][j];
10.    for (int i = 0; i < N; i++) {
11.        for (int j = 0; j <= N; j++)
12.            cout << C[i][j] << " ";
13.        cout << endl;
14.    }
15. }
16.
```

Con base al código anterior conteste las siguientes preguntas:

1. Revise la lógica dentro del triple ciclo anidado. ¿La operación sobre C[i][j] produce correctamente el producto matricial acumulado? Explique qué debería ocurrir matemáticamente y en qué difiere el código actual.
2. Examine los ciclos de impresión. ¿Los rangos de iteración son coherentes con las dimensiones declaradas de la matriz? Identifique cualquier inconsistencia y corrija el código.
3. 3. Verifique manualmente el resultado esperado de multiplicar $A = \{\{1,2\},\{3,4\}\}$ y $B = \{\{5,6\},\{7,8\}\}$ con el código corregido. ¿Coincide con el producto matricial correcto?

Ejercicio 2

20pts

La función eliminarNodo busca un valor en una lista doblemente enlazada, desconecta el nodo encontrado y libera su memoria. Al recorrer la lista hacia atrás después de una eliminación, los datos obtenidos son incorrectos. Estudie el código con atención:

```
1. void eliminarNodo(Nodo** cabeza, int valor) {
2.     Nodo* actual = *cabeza;
3.     while (actual != nullptr && actual->dato != valor)
4.         actual = actual->siguiente;
5.     if (actual == nullptr) return;
6.     if (actual->anterior != nullptr)
7.         actual->anterior->siguiente = actual->siguiente;
8.     else
9.         *cabeza = actual->siguiente;
10.    if (actual->siguiente != nullptr)
11.        actual->siguiente->anterior = actual->siguiente;
12.    delete actual;
13. }
14.
```

Con base al código anterior conteste las siguientes preguntas:

1. . Analice cómo se actualiza el puntero anterior del nodo sucesor tras la desconexión. ¿Apunta al nodo correcto? Trace un ejemplo con tres nodos [A↔B↔C] eliminando B para verificar el estado final de los punteros.
2. Considere todos los casos borde posibles: nodo al inicio, al final y único en la lista. ¿El código los maneja correctamente en todos los casos? Identifique cualquier situación no cubierta.
3. Reescriba la función completa con todos los ajustes necesarios para que funcione correctamente en todos los escenarios.